▶ 1분 미리보기

# Chapter 5. 비용 효율적으로 쓰기

같은 일을 더 적은 토큰으로 (Claude Code v2.1.319, 2026-11-08 리허설 검증)

**Choi, WooHyung**
Prin. Solutions Architect
AWS Korea

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 강의와 데모 20분, 랩 38분

여덟 가지 원리, 그리고 랩 TASK 01부터 04까지

CHAPTER 05 / 60분

**컨텍스트 엔지니어링**

먼저 보기, 지침 줄이기, 출력 필터, 캐시와 압축, 모델과 사고량

13:00 시작

LAB / Ch5 Lab

**TASK 01부터 04**

기준선 재기, 출력 필터 훅, CLAUDE.md 다이어트, 압축 시점

강의 뒤 바로

MORE

**더 알아보기**

자주 묻는 세부 내용은 마지막에 모아 둡니다

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 05 / 13:00

# 컨텍스트 엔지니어링

같은 일을 더 적은 토큰으로

01
측정 먼저

02
항상 실리는 지침 줄이기

03
구조 정보, 출력 필터, 도구 지연 로드

04
캐시와 압축(compaction)

05
모델과 사고량

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 출발점은 하루 13달러

엔터프라이즈 배포의 개발자 1인당 평균 비용

BASELINE

**$13**

활성일 하루, 개발자 1인당 평균

**월 $150에서 $250**
엔터프라이즈 배포에서 개발자 1인당 평균

**90%는 활성일 $30 미만**
대부분의 사용자는 하루 30달러를 넘지 않습니다

**우리 팀의 기준선은 직접 측정**
파일럿에서 Ch6의 계측으로 잽니다

출처: code.claude.com/docs/en/costs

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 원리 1, 측정 먼저

/context와 /usage로 무엇이 자리를 차지하는지

## /usage, Session (예시 출력)

```
Total cost:            $0.55
Total duration (API):  6m 20s
Usage by model:
  claude-sonnet-5-5:  1.2k input, 5.3k output,
                      940.0k cache read, 50.0k cache write
Prompt cache (main):   14 requests, 91% of input tokens
                       from cache, 2 misses, warm (1h TTL)
```

`/context`
**항목별 컨텍스트 점유**

`/usage`
**세션 비용과 캐시 적중률**

`miss`
**캐시 미스에는 추정 원인이 표시됨**

`금액`
**금액은 추정치, 정산은 AWS 청구서**

Total cost는 추정치입니다. 실제 정산은 AWS 청구서이고,
Ch6에서 계약 단가로 맞추는 법을 봅니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 같은 일을 더 적게, 여덟 가지 원리

본론은 공식 기능, 커뮤니티 도구는 사례로만

원리 1
**먼저 본다**

원리 2
**항상 실리는 지침은 짧게**

원리 3
**탐색 대신 구조 정보**

원리 4
**큰 출력은 들어오기 전에**

원리 5
**도구는 필요할 때만**

원리 6
**캐시를 깨지 않는다**

원리 7
**압축은 내가 정한 때에**

원리 8
**작업에 맞는 모델, 사고량**

## 작업에 맞는 모델, 사고량

| | |
|---|---|
| 공식 기능 | /model, /effort, Subagent model |
| 커뮤니티 | 없음 |

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 원리 2, 항상 실리는 지침은 짧게

매 요청에 실리는 것과 필요할 때 오는 것

STEP 4 **SessionStart 훅으로 짧은 규칙만**

꼭 항상 알아야 하는 짧은 규칙은 **SessionStart 훅**으로 세션 시작에 주입할 수 있습니다.




● 항상 실림 ● 요청 시 ● 세션 시작 훅

**커뮤니티 사례, 비공식**

효과 수치는 제작자 자체 보고

```
Ponytail
# SessionStart 훅으로
# 약 100줄의 YAGNI(지금 필요 없는 것은 만들지 않기)
규칙을
# 주입하는 플러그인
# 제작자 자체 보고 수치로 비용 약 10% 감소
```

사례는 원리를 보여 주는 용도입니다. 도입 판단은 Part 1 끝의 세 가지 질문으로 합니다.

- 세션 시작에 짧은 규칙 주입
- 길게 쓰면 CLAUDE.md와 같은 비용
- 커뮤니티 사례: Ponytail (비공식)

‹ 4 / 4 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 원리 3, 탐색 대신 구조 정보

grep과 파일 여러 개 읽기를 한 번의 질의로

OFFICIAL

### Code intelligence 플러그인

언어 서버로 정의로 이동합니다.

편집 후 타입 오류를 자동으로 보고해, 불필요한 파일 읽기가 줄어듭니다.

OFFICIAL

### 코드베이스 개요 스킬

아키텍처, 디렉터리, 명명 규칙을 스킬로 둡니다.

탐색에 쓰는 토큰을 줄입니다.

COMMUNITY / 비공식

### Graphify

코드를 로컬 AST로 파싱해 지식 그래프로 만들고 스킬, MCP로 제공합니다.

작은 저장소에서는 효과가 거의 없습니다.

> 커뮤니티 사례
>
> 효과 수치는 제작자 자체 보고입니다. 같은 효과를 공식 기능으로 낼 수 있는지 먼저 봅니다.

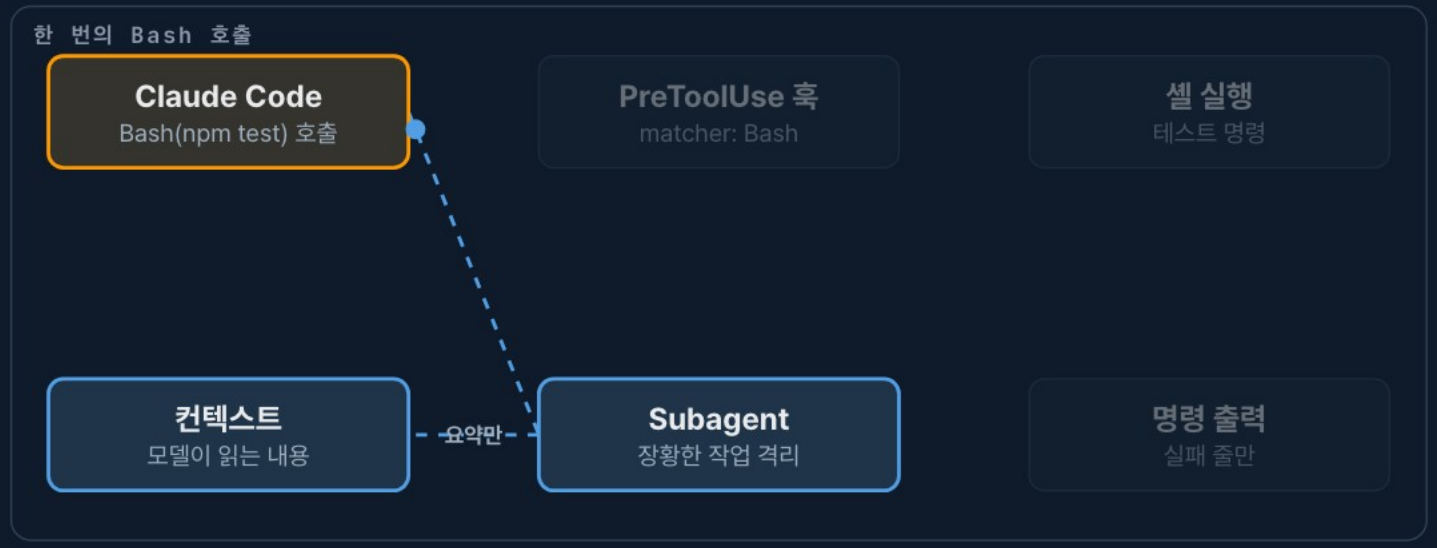
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 원리 4, 큰 출력은 들어오기 전에

공식 예제 기반: 테스트 출력을 실패 줄만 남기기

STEP 5 **장황한 작업은 Subagent로 격리합니다**

출력을 줄이기 어려운 장황한 작업은 **Subagent**에 맡깁니다. Subagent가 읽고, 메인 컨텍스트에는 **요약만** 돌아옵니다.




● Claude Code와 셸 ● 훅 ● 큰 출력 ● 줄어든 출력 ● 컨텍스트와 Subagent

**커뮤니티 사례, 비공식**

효과 수치는 제작자 자체 보고

```
Headroom
# 요청 경로에 프록시를 두고
# 도구 출력과 로그를 압축
# 주의: 프록시가 모든 프롬프트를 지나감
```

금융권에서는 **프롬프트가 어디를 지나가는지**가 먼저입니다.

- Subagent가 긴 출력을 대신 읽음
- 메인에는 요약만
- 커뮤니티 Headroom: 프록시가 모든 프롬프트를 지나감, 주의

‹ 5 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 훅 하나로 컨텍스트 절감

DEMO 7분, 출력 필터 훅 적용 전후 비교

DEMO / 7분

| | |
|---|---|
| 1 | 실패 테스트가 섞인 저장소에서 "npm test 돌려서 실패 원인 찾아줘" |
| 2 | /context와 /usage 기록 |
| 3 | filter-test-output.sh 등록 후 /clear, 같은 요청 반복 |
| 4 | 두 번의 컨텍스트 크기와 비용 비교 |

강사 데모입니다. 참가자는 Lab 포털의 같은 스크립트로 따라 해 볼 수 있습니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 원리 5, 도구는 필요할 때만

MCP 서버는 공짜가 아닙니다

DEFERRED

### 도구 정의 지연 로드

기본값은 도구 이름과 서버 지침만 싣고, 쓸 때 스키마를 tool search로 로드합니다.

게이트웨이 뒤에서도 기본 켜짐 (v2.1.304+).

TRIM

### 안 쓰는 서버 끄기

/mcp로 서버 목록을 확인합니다.

쓰지 않는 서버는 비활성화합니다.

CLI FIRST

### CLI 우선

gh, aws 같은 CLI는 도구 목록을 미리 싣지 않습니다.

> 커뮤니티 사례 / 비공식
>
> **REA**(Reverse Engineer Anything)는 170여 개 MCP 도구를 가진 역공학 도구입니다. 도구 목록이 거대해 지연 로드가 왜 기본값인지 보여 주는 사례입니다. 비신뢰 바이너리는 VM과 법무 검토를 거칩니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 한 시간 비운 뒤 첫 메시지는?

QUICK CHECK, 손을 들어 답해 주세요

## 1시간 넘게 자리를 비운 뒤 보내는 첫 메시지는?

A 캐시에서 읽어 저렴하다

B 캐시 미스로 전체 컨텍스트를 다시 처리한다

C 자동으로 /clear 된다

> 정답 B. 기본 캐시 수명은 5분입니다. 긴 세션일수록 비용이 큽니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 원리 6, 캐시 보존

/clear와 /compact는 비용이 다릅니다

### /clear

- 새 세션 시작, 비용 0
- 관련 없는 작업으로 넘어갈 때
- /rename 후 /resume으로 돌아오기

### /compact

- 대화를 읽어 요약하는 것 자체가 큰 요청
- 이어서 작업해야 할 때만
- 보존할 것을 지시: /compact Focus on ...
- CLAUDE.md에 압축 지침을 둘 수 있음

**5분**
기본 캐시 수명

**1시간**
1시간 옵션, 더 비쌈

**미스 원인**
/usage에 표시

**Bedrock**
모델, 리전별 지원 확인

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 원리 7, 압축은 내가 정한 때에

대화가 차면 Claude Code는 오래된 기록을 요약합니다

자동
**자동 압축**

수동
**/compact [지시]**

부분
**/rewind 부분 요약**

자리 비움
**Idle compaction**

전환
**/clear**

**/clear**

| | |
|---|---|
| 언제 | 관련 없는 작업으로 전환 |
| 결과 | 비용 0, 연속성 없음 |

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 가득 차기 전 선제 압축

auto-compact window 조정과 설정 우선순위

Claude Code 세션, Terminal

```
# 현재 값과 출처 확인
/autocompact
# 400K에서 압축 (모델별로 사용자 설정에 저장, v2.1.288+)
/autocompact 400k
# 모델 기본값으로 복귀
/autocompact auto
# 이번 세션만
claude --autocompact 500k
# 조직 기본값 (managed-settings.json)
{ "autoCompactWindow": 400000 }
```

> 같은 파일에서는 modelSettings.<model> 값이 우선합니다.
> 범위는 100K에서 1M 또는 "auto", 실제 시점은 설정값과 모델 최대 컨텍스트 중 작은 쪽입니다.

1 환경 변수 (최우선)
`CLAUDE_CODE_AUTO_COMPACT_WINDOW`

2 --autocompact 실행 플래그

3 settings: managed가 사용자 /autocompact를 덮음




대화가 400K에 닿으면 자동 압축이 시작됩니다.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 창을 줄이면 항상 쌀까?

Claude Code도 기본값 "auto"를 권하는 이유

### 줄어드는 것

- 압축 전까지 매 요청의 입력 토큰
- 캐시가 식은 뒤 다시 읽는 양
- 1M 모델의 긴 세션에서 효과가 큼

### 늘어나는 것, 잃는 것

- 압축 횟수: 한 번이 대화 전체를 읽는 요청
- 압축 직후 캐시 재구축
- 손실 요약: 초반 제약, 정확한 오류가 빠질 수 있음
- 기본값을 바꾸면 긴 세션 재개 시 사용량이 늘 수 있다는 공식 경고

**1순위**
작업 단위 끝에 캐시가 따뜻할 때 /compact

**2순위**
긴 세션 팀만 창을 400K에서 500K로

**검증**
OTel token.usage로 전후 비교

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 압축 후에 무엇이 남나

공식 문서의 What survives compaction

/compact 실행해 보기 압축 후

| | | |
|---|---|---|
| 시스템 프롬프트, 출력 스타일 | 유지 | 그대로 적용 |
| 루트 CLAUDE.md, auto memory, plan 모드의 계획 | 재주입 | 디스크에서 다시 주입 |
| 최근 읽거나 고친 파일 | 재주입 | 최근 수정 순으로 최대 5개 다시 읽음 (5,000 토큰 넘으면 경로만) |
| 사용한 스킬 본문 | 재주입 | 스킬당 5,000, 합계 25,000 토큰, 오래된 것부터 제외 |
| paths: 규칙, 하위 폴더 CLAUDE.md | 사라짐 | 요약되어 사라지고, 해당 파일을 다시 읽을 때 재로드 |
| SessionStart 훅 (matcher: compact) | 재실행 | 다시 실행되어 결과가 추가됨 |

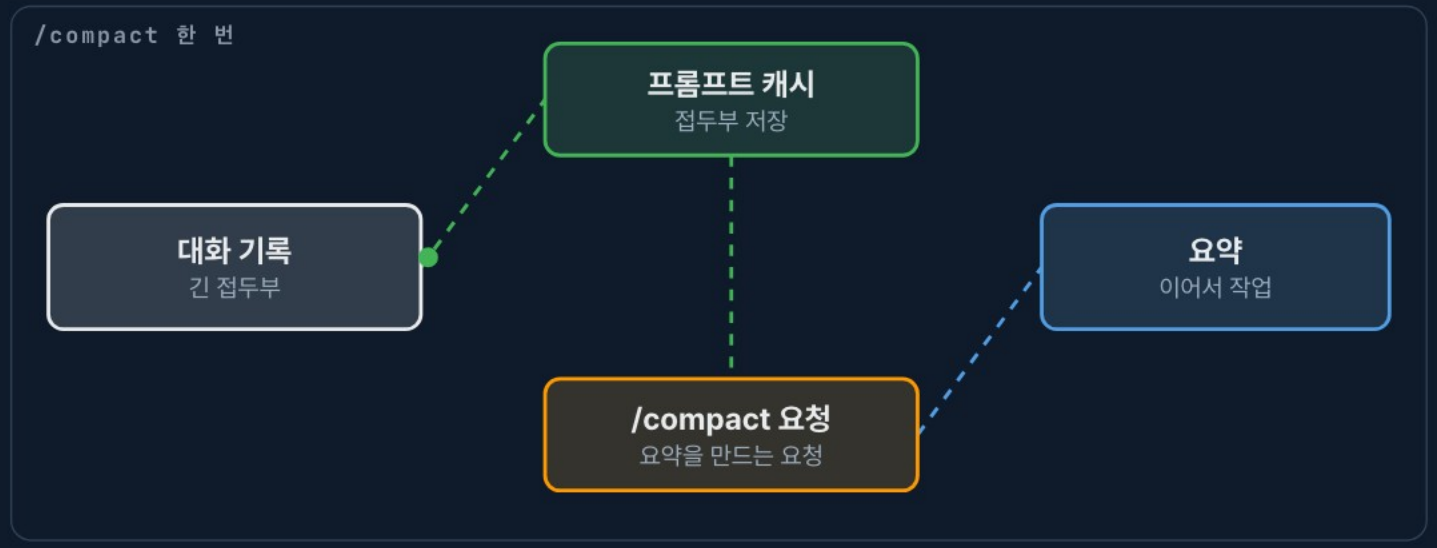
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 자리 비우기 전에 /compact

/compact도 대화 전체를 읽는 요청, 캐시가 따뜻할 때와 식었을 때

STEP 3 **떠나기 전에 정리합니다**

추천은 **Compact before you step away**입니다. 이어서 할 일이면 /compact, 관련 없는 일이면 /clear, 이미 캐시된 지점으로는 /rewind로 돌아갑니다.




캐시 적중 · 비캐시 재처리 · 만료된 캐시 · 요약 요청

**추천**

Compact before you step away

```
/compact    # 이어서 할 일이면
/clear      # 관련 없는 일이면
/rewind     # 이미 캐시된 지점으로 복귀
/usage      # expected rebuild 따로 집계
```

회의나 점심 전에 습관처럼 한 줄.

- 자리 비우기 전에 /compact 또는 /clear
- /usage는 expected rebuild로 따로 집계
- /rewind: 이미 캐시된 지점으로 복귀

‹ 3 / 3 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 같은 기능, 접속 방식별 차이

Bedrock에서 캐시 수명과 idle compaction

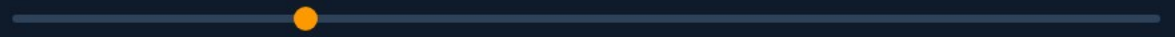
구독
**Claude 구독 (플랜 사용량 안)**

기본
**API 키, Amazon Bedrock**

옵션
**Bedrock + 1시간 캐시 옵션**

게이트웨이
**Claude Apps Gateway 경유**

| | |
|---|---|
| 메인 대화 기본 TTL | 5분, 정책으로 1시간 옵트인 (서빙하는 모든 업스트림 모델이 지원해야 함) |
| Idle compaction | 1시간 옵트인 시 같은 조건 |
| 자리 비운 시간 | 30분 (예시) |



**캐시가 만료됐습니다. 첫 요청은 전체 컨텍스트를 다시 처리합니다.**

1시간 옵션은 더 비쌉니다. 자리 비운 시간은 설명용 예시입니다.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 압축 설계를 조직 기본값으로

managed settings로 창 크기, 캐시 수명, 압축 후 규칙을 고정

STEP 4 **압축 전에 감사 기록을 남깁니다**

**PreCompact** 훅은 압축 직전에 실행됩니다. **exit 2**로 압축을 막을 수 있고, 감사용으로 대화를 보관합니다.




관리자가 강제 · 비용 설정 · 훅

**압축 전 감사**

`managed-settings.json, hooks`

```
"PreCompact": [
  { "hooks": [{ "type": "command",
      "command": "/opt/audit/save-
transcript.sh" }] }
]
```

요약되기 전의 원문을 회사 정책에 따라 보관합니다.

- 압축 직전 실행
- exit 2로 압축 차단
- 감사용 보관

‹ 5 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 압축을 내가 정한 때에, 15분

LAB, Chapter 5 TASK 04

| | |
|---|---|
| 1 | /context, /usage로 압축 전 기록 |
| 2 | /compact 테스트 실패 원인과 남은 할 일만 남겨줘 |
| 3 | CLAUDE.md에 # Compact instructions 추가 |
| 4 | SessionStart compact 훅으로 사내 규정 재주입 확인 |
| 5 | /autocompact 400k로 창 조정, 저장 위치 확인, auto로 복귀 |
| 6 | /config에서 Idle compaction 항목과 Bedrock TTL 확인 |

Ch5 랩: TASK 01 기준선 5분, 02 출력 필터 훅 10분, 03 CLAUDE.md와 모델 8분, 04 압축 15분. Hands-on Lab 포털에서 Chapter 5를 엽니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 원리 8, 작업에 맞는 모델과 사고량

사고 토큰은 출력 단가로 청구됩니다

MODEL

**/model**

대부분의 코딩은 Sonnet

복잡한 설계만 Opus

EFFORT

**/effort**

단순 작업은 effort를 낮춤

5.5 세대와 Fable은 thinking을 끌 수 없음

SUBAGENT

**model: haiku**

단순 Subagent는 작은 모델

고정 예산 모델은 MAX_THINKING_TOKENS

> 사고(thinking) 토큰은 **출력 단가**로 청구됩니다. 모델과 사고량을 작업 크기에 맞추는 것이 마지막 원리입니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 커뮤니티 도구 앞의 세 가지 질문

도입 전 5분 검토

QUESTION 1

### 어느 원리를 쓰는가

여덟 가지 원리 중 무엇이고, 어느 확장 지점(훅, 스킬, MCP, 프록시)에 붙는가

QUESTION 2

### 공식 기능으로 되는가

CLAUDE.md, 스킬, 필터 훅, code intelligence로 같은 효과를 낼 수 있는가

QUESTION 3

### 데이터는 어디로 가는가

로컬 처리인지, 제3자로 가는지. 도입한다면 사내 마켓플레이스에 고정 버전으로 (Ch2, Ch4)

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 덜 쓰는 세 가지

Chapter 5 Key Takeaways, 16:50 Chapter 6으로 이어짐

**01**

먼저 본다: /context, /usage, 캐시 적중률
캐시를 지키고, 압축 시점과 effort는 내가 정한다

**02**

항상 실리는 지침은 짧게, 큰 출력은 들어오기 전에

**03**

커뮤니티 도구는 원리로 읽고 공식 기능으로 대체 가능한지 본다

Chapter 6은 16:50, 조직이 보고 막는 법입니다. 그 전 Capstone 시간에도 /usage를 한 번씩 확인해 보세요.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 자주 받는 질문

더 알아보기, MCP 비용, 세션 위생, 모델 배분

1
**MCP 도구 정의 비용**

2
**세션 위생**

3
**모델 배분 예시**

### 역할별로 모델을 고정

```
# .claude/agents/log-summarizer.md
---
name: log-summarizer
description: 긴 테스트, 빌드 로그를 실패 원인 중심으로 5줄 요약한다. 로그
분석 요청 시 사용.
tools: Read, Grep
model: haiku
---
실패한 테스트 이름, 첫 오류 메시지, 의심 파일만 보고한다.
```

장황한 로그는 작은 모델이 읽고, 메인에는 5줄만 돌아옵니다. 메인 모델은 판단에 집중합니다. Bedrock 계정에 Haiku가 활성화되어 있는지 확인하세요.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Thank You

다음은 14:00 Capstone Round 2이고, 16:50에 Chapter 6. 조직의 비용 보고 통제하기로 이어집니다. 질문은 Slack #20261116-claude-code-workshop-for-fsi 채널에서 언제든 받습니다.

← 목차로 돌아가기 Chapter 6 (16:50) →

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.